INTRODUCTION ABOUT THE PROJECT

In today's digitally connected world, the demand for online entertainment and video content consumption has grown exponentially. While major streaming platforms exist, building a custom full-stack streaming solution provides invaluable experience in modern web development. The need for a self-hosted, fully customizable movie streaming platform with content management, user authentication, and community reviews creates a rich learning and practical opportunity. This is where **Movie Streaming Application** comes in.

Movie Streaming Application bridges the gap between users and digital entertainment by offering a comprehensive, feature-rich video streaming platform. Designed with a focus on performance, scalability, and user engagement, the platform provides a streamlined approach to discovering and watching movies and videos. Users can explore a curated library of movies across various genres, with features like personalized movie feeds, advanced search and filtering, and the ability to rate and review content. Administrators can manage the entire movie catalog, handle user movie requests, and oversee content through a powerful admin dashboard.

The goal of Movie Streaming Application is to create a one-stop solution for movie discovery and video streaming. By leveraging modern web technologies and best practices, the platform not only provides a seamless user experience but also ensures that high-quality content is easily accessible. The system supports YouTube-based video embedding, allowing users to watch movies and episodes directly within the platform without leaving the application.

As digital streaming becomes the dominant form of entertainment consumption worldwide, Movie Streaming Application is built to meet these evolving needs. Its responsive design,

dynamic search capabilities, and interactive features—such as real-time rating and review systems, video quality selection, and progress tracking—make it a powerful tool for users seeking to discover, watch, and review their favorite movies and video content from the comfort of any device.

From a technical perspective, the platform's development is rooted in modern full-stack web development principles, utilizing the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)** to create a scalable, single-page application (SPA). The project follows an iterative development approach, ensuring features like authentication, movie management, rating systems, and admin dashboards are implemented incrementally. The back-end architecture is built using a modular Controller pattern with RESTful APIs and TypeScript, ensuring type safety, security, maintainability, and ease of future enhancements.

By combining ease of use, performance, and a rich content library, Movie Streaming Application seeks to stand out as a premier digital streaming hub. Whether a user is looking for the latest movies, top-rated films, or wishes to submit a request for new content to be added, Movie Streaming Application offers the tools and resources needed to enjoy a complete streaming experience.

OBJECTIVE AND SCOPE OF PROJECT

The **Movie Streaming Application** has the following specific objectives:

- **To build a full-featured movie streaming platform** where users can browse, search, and stream movies and video content organized by genre, release year, cast, and director.
- **To implement a secure user authentication system** using JWT-based authentication and bcrypt password hashing, distinguishing between regular users and administrators with role-based access control.
- **To enhance user engagement** through interactive features such as a movie rating and review system (1–5 stars), movie carousels, toast notifications, and a movie request system allowing users to suggest new content to administrators.
- **To provide a comprehensive admin dashboard** enabling administrators to create, update, and delete movies, manage genres, moderate user reviews, and process movie requests submitted by users.
- **To facilitate easy content discovery** through advanced search functionality and filtering options, allowing users to find movies by title, genre, release year, cast, director, or rating, with separate feeds for new movies, top-rated movies, and random picks.
- **To ensure accessibility and ease of use** via a responsive, mobile-friendly interface built with Tailwind CSS v4 and React 19, delivering a seamless experience across desktops, tablets, and smartphones.

The scope for developing the **Movie Streaming Application** includes:

- **User Management:** Implementing secure JWT-based authentication (Register/Login/Logout) with bcrypt password hashing, profile management, and role-based access control distinguishing regular users from administrators.
- **Movie Content Management:** Developing a robust admin system for administrators to create, update, and delete movies with full metadata (name, year, genre, cast, director, cover image), and manage video episodes via YouTube embedding.
- **Rating and Review System:** Integrating a movie rating system (1–5 stars) and text-based review system, with duplicate review prevention and admin ability to moderate and delete reviews.
- **Advanced Search & Filtering:** Creating endpoints to retrieve movies sorted by "New Releases," "Top Rated," and "Random Picks," with filtering by genre, year, rating, and duration, and a dedicated search functionality.
- **Responsive User Interface:** Utilizing the **Tailwind CSS v4** utility framework with React 19 to ensure the platform adapts dynamically to different screen sizes, providing a modern and visually consistent streaming interface.
- **Data Security:** Implementing secure backend architecture using Node.js and MongoDB to protect user data and ensure safe interactions.

Introduction of frontend and backend tools

A. INTRODUCTION OF HTML AND CSS

HTML (Hypertext Markup Language) and **CSS (Cascading Style Sheets)** are the foundational technologies of web development. In the context of **Movie Streaming Application**, these technologies are utilized within the React.js framework (via JSX) to build a dynamic and visually engaging Single Page Application (SPA).

HTML (Hypertext Markup Language): HTML serves as the structural backbone of the application. In this project, HTML is implemented using **TSX (TypeScript XML)**, a TypeScript extension for React. This allows us to write HTML structures with full type safety within TypeScript code, defining the layout of components such as the MovieCard, VideoPlayer, Admin Dashboard, and Genre filters. It categorizes elements like search inputs, rating stars, review forms, and navigation links, providing the semantic structure necessary for accessibility and SEO.

CSS (Cascading Style Sheets) & Tailwind CSS v4: CSS is responsible for the visual presentation of the platform. It controls the layout, colors, fonts, and spacing, ensuring a pleasing aesthetic.

- **Custom CSS:** The project leverages `Tailwind CSS v4` utility classes directly in component files, with additional custom stylesheets (`App.css`, `index.css`) to define specific visual identities, such as the movie card hover effects, video player styling, and the carousel component animations.

- **Tailwind CSS v4 Framework:** To accelerate development and ensure responsiveness, the project integrates **Tailwind CSS v4** (via `Tailwind CSS v4`). This utility-first framework provides atomic CSS classes that allow developers to build fully custom, responsive layouts by composing utility classes directly in component markup. It automatically optimizes the streaming platform's layout based on the user's device, ensuring a consistent cinematic experience whether on a desktop or a mobile phone.

Together, HTML (via TSX) and CSS (enhanced by Tailwind CSS v4) enable the rapid development of a visually rich, responsive streaming interface that separates content structure from visual design, adhering to modern web standards and delivering a Netflix-like user experience.

B. INTRODUCTION OF JAVASCRIPT (FULL STACK)

JavaScript is a versatile and powerful programming language that serves as the core technology for the entire **Movie Streaming Application** stack. Unlike traditional web development where JavaScript was limited to the browser, this project utilizes **Full Stack JavaScript**, employing it on both the client-side (React.js) and the server-side (Node.js).

Key Features of JavaScript in this Project:

Unified Language (Full Stack Development): One of the most significant advantages of this project is the use of JavaScript across the entire technology stack (MERN). This allows for seamless data flow (JSON) between the frontend and backend, reducing context switching and simplifying the development process.

Client-Side Scripting (React.js): On the frontend, JavaScript is used to build interactive user interfaces. It handles:

- **DOM Manipulation:** Updating the view dynamically without reloading the page (e.g., when a user votes on a poll or posts a comment).
- **State Management:** Using tools like **Redux Toolkit** to manage application state (user authentication, fetched topics, UI loading states).
- **Event Handling:** Responding to user actions such as button clicks, form submissions, and keyboard shortcuts and dynamic interactions (e.g., rating selection, review submission).

Server-Side Scripting (Node.js): On the backend, JavaScript runs on the server via **Node.js**. It handles:

- **API Logic:** Processing requests from the frontend (e.g., fetching topics, registering users).
- **Database Interaction:** Communicating with MongoDB via **Mongoose** to store and retrieve data.
- **Authentication:** Verifying user identities and managing secure sessions (JWT).

Asynchronous Programming: The project heavily relies on JavaScript's asynchronous features (`async/await`, `Promises`). This ensures non-blocking operations, such as waiting for database queries or API responses without freezing the user interface, resulting in a highly performant application.

Object-Oriented & Functional Paradigms: JavaScript supports both object-oriented and functional programming styles. This project utilizes modern **ES6+ features** (Arrow Functions, Destructuring, Modules), allowing for clean, modular, and maintainable code structure across both the client and server environments.

C. INTRODUCTION OF NODE.JS

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser. Initially created by Ryan Dahl in 2009, Node.js was developed to push JavaScript beyond the client-side, allowing developers to use it for server-side scripting. Unlike PHP, which is a blocking, synchronous language, Node.js is built on the V8 JavaScript engine—the same high-performance engine that powers Google Chrome—converting JavaScript directly into native machine code.

Node.js operates on an event-driven, non-blocking I/O model, which makes it lightweight and efficient. Instead of creating a new thread for every user request (as traditional web servers do), Node.js uses a single-threaded event loop to handle thousands of concurrent connections. This architecture makes it uniquely suited for data-intensive, real-time applications like **Movie Streaming Application**, where features such as real-time content loading and user interactions are crucial.

While Node.js is primarily associated with backend web development, it is a versatile environment capable of powering command-line tools, desktop applications, and Internet of Things (IoT) devices. In the context of the MERN stack, Node.js serves as the backbone, hosting the Express.js 5 framework and managing interactions between the React 19 TypeScript frontend and the MongoDB database.

The Node.js ecosystem is supported by the Node Package Manager (NPM), which is the largest software registry in the world. This vast library of reusable packages allows developers to

integrate complex functionalities—such as authentication, image processing, and database connectivity—without reinventing the wheel.

As of 2024, Node.js is the technology of choice for high-traffic platforms like Netflix, Uber, and LinkedIn, proving its reliability and scalability. By allowing the use of a single language (JavaScript) on both the client and server sides, Node.js unifies web application development, streamlining the coding process and reducing the learning curve for developers.

Features of Node.js:

Asynchronous and Event-Driven: All APIs of the Node.js library are asynchronous, meaning they are non-blocking. A Node.js-based server never waits for an API to return data. The server moves to the next API after calling it, and a notification mechanism helps the server receive a response from the previous API call. This ensures high throughput and speed.

High Performance: Built on Google Chrome's V8 JavaScript Engine, Node.js compiles JavaScript code directly into native machine code. This results in incredibly fast execution speeds, making it superior for tasks requiring high computation or frequent database operations.

Single-Threaded but Scalable: Node.js uses a single-threaded model with event looping. This mechanism helps the server respond in a non-blocking way and makes the server highly scalable as opposed to traditional servers which create limited threads to handle requests. This allows **Movie Streaming Application** to handle a large number of concurrent users streaming video content simultaneously without crashing.

Unified Tech Stack (Full Stack JavaScript): One of the distinct advantages of Node.js is that it allows developers to write JavaScript for both the client-side (Frontend) and server-side (Backend). This eliminates the need to context-switch between languages (like PHP and JS), leading to cleaner, more consistent code and faster development cycles.

Cross-Platform: Like PHP, Node.js is completely cross-platform. It can run seamlessly on Windows, Linux, macOS, and Unix systems. This ensures that the application can be deployed on virtually any server environment without modification.

No Buffering: Node.js applications output the data in chunks rather than buffering the entire data set. This is particularly useful for features like uploading large images or streaming video content, providing a smoother user experience.

Active Community and NPM: Node.js boasts a massive, active community of developers. Through NPM (Node Package Manager), developers have access to millions of open-source libraries and modules. This community support ensures that security patches, updates, and new features are continuously available, keeping the platform secure and modern.

JSON Support: Since Node.js is JavaScript-based, it handles JSON (JavaScript Object Notation) natively. This is a significant advantage over other backend languages, as JSON is the standard format for API communication (REST APIs) and is also the native storage format of MongoDB, ensuring seamless data flow throughout the application..

D. INTRODUCTION OF MONGODB

What is a Database? A database is a specialized application designed to store, manage, and organize data in a structured manner. It provides distinct APIs that allow users to create, access, manage, search, and replicate the data it contains. While traditional systems used tabular data structures, modern web applications like **Movie Streaming Application** often require more flexible data storage solutions to handle complex, unstructured data efficiently.

NoSQL Database Management System Unlike Relational Database Management Systems (RDBMS) which organize data into rigid tables, MongoDB is a **NoSQL (Not Only SQL)** database. It is classified as a **Document-Oriented Database**. Instead of rows and columns, it stores data in flexible, JSON-like documents. This structure allows for:

- **Dynamic Schemas:** Data structures can change over time without modifying the entire database.
- **High Scalability:** Designed to handle massive amounts of data across many servers (horizontal scaling).
- **Hierarchical Data Storage:** Complex data structures (like arrays and nested objects) can be stored within a single document, reducing the need for complex joins.

MongoDB Terminology Before exploring the specifics of MongoDB, it is important to understand the key terminology used in Document-Oriented databases and how they differ from traditional RDBMS:

- **Database:** A physical container for collections. Each database gets its own set of files on the file system.

- **Collection:** A collection is a grouping of MongoDB documents. It is the equivalent of an RDBMS **Table**. A collection exists within a single database.
- **Document:** A document is a set of key-value pairs. It is the equivalent of an RDBMS **Row** (or record). Documents have dynamic schemas, meaning documents in the same collection do not need to have the same set of fields.
- **Field:** A field is a key-value pair in a document. It is the equivalent of an RDBMS **Column**.
- **_id:** The primary key for every document. MongoDB automatically assigns a unique `_id` field to every document to ensure it can be uniquely identified.
- **BSON:** Binary JSON. This is the format MongoDB uses to store documents. It extends the JSON model to provide additional data types and to be efficient for encoding and decoding.
- **Embedding:** The practice of nesting data (such as comments inside a topic) directly within a document. This improves read performance by keeping related data together.
- **Sharding:** The process of storing data records across multiple machines to support big data deployments.

What is BSON and MQL? While SQL is used for relational databases, MongoDB uses **MQL (MongoDB Query Language)** and stores data in **BSON (Binary JSON)**.

BSON (Binary JSON): BSON is a binary-encoded serialization of JSON-like documents. It supports embedding of documents and arrays within other documents and records. BSON also contains extensions that allow representation of data types that are not part of the JSON spec, such as `Date` and `BinData`.

CRUD Operations (The Foundation of MQL): Similar to the DML (Data Manipulation Language) in SQL, MongoDB provides powerful operators to interact with data:

1. **Create:** `insertOne()`, `insertMany()` – Used to add new documents to a collection.
2. **Read:** `find()`, `findOne()` – Used to query data. MongoDB supports rich queries, including searching by field, range queries, and regular expression searches.
3. **Update:** `updateOne()`, `updateMany()` – Used to modify existing documents. It supports atomic operators like `$set`, `$inc` (increment), and `$push` (add to array).
4. **Delete:** `deleteOne()`, `deleteMany()` – Used to remove documents from a collection.

MongoDB MongoDB is a widely used, open-source, non-relational database management system developed by MongoDB Inc. It is designed for modern application developers and the cloud era. As a document database, MongoDB makes it easy for developers to store structured or unstructured data.

MongoDB is a core component of the **MERN Stack** (MongoDB, Express, React, Node.js) used in this project. It connects seamlessly with Node.js and React because data is passed as JSON objects throughout the entire application lifecycle.

Advantages of MongoDB:

- **Schema-less:** MongoDB is a schema-less database, which implies that one collection can hold different documents. This flexibility allows for rapid iteration during the development of features like movie video episodes and genre management in the Movie Streaming Application.

- **Performance:** MongoDB stores data in internal memory (RAM) for faster access. It is optimized for high read/write throughput, making it ideal for real-time applications.
- **Scalability:** MongoDB supports horizontal scaling through **Sharding**, allowing the database to handle growing amounts of data by distributing it across multiple servers.
- **Deep Querying:** MongoDB supports dynamic queries on documents using a document-based query language that is nearly as powerful as SQL.
- **High Availability:** MongoDB's replication feature (Replica Sets) provides automatic failover and data redundancy, ensuring the application remains online even if a server fails.
- **JSON Compatibility:** Since the entire project is built on JavaScript (React frontend, Node backend), MongoDB's native JSON-like storage format eliminates the need for complex data translation layers, simplifying the codebase.

DEFINITION OF PROBLEM

The current landscape of movie streaming and content management faces several challenges related to content organization, user experience, and system performance. While users utilize various platforms for watching movies, they often lack a unified platform that combines streaming, user reviews, content management, and request handling in one place. These problems impact the quality of the user experience and efficiency of content delivery. Below are the primary problems Movie Streaming Application addresses:

1. Fragmented Communication Channels:

- **Issue:** Movie and video content is scattered across multiple platforms with no unified system for discovery, rating, or requesting new content.
- **Impact:** Users cannot easily track what they have watched, rate content, or request movies in one centralized place.

2. Lack of Structured Knowledge Organization:

- **Issue:** Without proper genre classification and metadata, different types of movies (e.g., Action vs. Documentary vs. Anime) are difficult to categorize and discover.
- **Impact:** Users struggle to find relevant content quickly, leading to poor discovery rates and reduced platform engagement.

3. Content Quality and Validation:

- **Issue:** Without a structured rating and review system, users cannot evaluate movie quality or distinguish popular content from poor-quality releases.

- **Impact:** Users may waste time on low-quality content. The lack of a community rating system (1–5 stars) makes it hard to identify top-rated movies.

4. User Engagement and Interactivity:

- **Issue:** Static movie listing websites without interactive features fail to engage modern streaming audiences.
- **Impact:** Low participation rates. The platform needs interactive features like **Polls** and **Real-time Commenting** to foster active community participation.

5. Database Scalability and Performance:

- **Issue:** As the movie catalog and user review data grows, retrieving and serving content efficiently becomes challenging.
- **Impact:** Slow loading times can deter users. The system needs optimized MongoDB queries, aggregation pipelines, and efficient data fetching to handle high volumes of movie and review data.

6. Mobile Accessibility:

- **Issue:** Users primarily access streaming platforms via smartphones and tablets.
- **Impact:** A desktop-only interface would alienate the majority of users. The platform requires a fully responsive design built with Tailwind CSS to ensure seamless video streaming and browsing on any device.

7. Data Security and User Privacy:

- **Issue:** Handling user credentials and sensitive profile data requires strict security measures.
- **Impact:** Potential vulnerability to unauthorized access. The system must implement robust authentication (JWT) and password encryption (Bcrypt) to safeguard user identities and prevent unauthorized content management.

SYSTEM ANALYSIS

System analysis for **Movie Streaming Application** involves understanding the requirements of students and the academic community. The system must provide a seamless user experience while ensuring data security and efficient management of discussion topics. The analysis phase includes identifying key functionalities that will make the forum a vibrant hub for learning.

1. Problem Identification:

- Current movie streaming solutions lack integrated content management, review systems, and user request handling in a single platform.
- Users struggle to discover movies by genre, rating, or year without advanced filtering capabilities.
- There is no unified platform that combines streaming, user reviews, admin content management, and movie request workflows in a single MERN-based application.

2. User Requirements:

- **Regular Users:** Need a platform to browse movies, search by genre/year/rating, watch embedded YouTube video content, submit movie ratings and reviews, and request new movies to be added.
- **Administrators:** Require features to create and manage the complete movie catalog, handle movie requests from users, moderate reviews, upload movie thumbnails and cover images, and manage genre classifications.

3. System Requirements:

- **Functional:**
    - User Authentication (Register/Login/Logout) with JWT and bcrypt.
    - Movie Management (Create, Read, Update, Delete) with full metadata including genre, cast, director, and YouTube video episodes.
    - Movie Rating and Review system (1–5 stars, one review per user per movie).
    - Genre management for movie categorization and admin content organization.
    - Movie Request system allowing users to submit requests visible on the admin dashboard.
- **Non-Functional:**
    - **Scalability:** The ability to handle growing numbers of users and topics via MongoDB's document model.
    - **Performance:** Fast page loads using React 19's optimized rendering, Vite build tooling, and Single Page Application (SPA) architecture with TypeScript for type safety.
    - **Security:** Protection against common web vulnerabilities (XSS, NoSQL Injection) with secure cookie-based JWT session management and role-based authorization middleware.

4. System Architecture:

- The system is based on the **MERN architecture**, separating the frontend (Client) from the backend (Server).

- **Frontend (View):** Developed using **React 19 with TypeScript** and **Redux Toolkit** with Tailwind CSS v4, providing a dynamic, responsive streaming interface with movie carousels, search, and video player components.
- **Backend (Controller/Model):** Developed using **Node.js** and **Express.js**, managing business logic, API routes, and file uploads with Multer. Full API documentation is provided via Swagger/OpenAPI integration.
- **Database:** Uses **MongoDB**, a NoSQL database, to store unstructured data like discussion threads and user profiles efficiently.

5. Data Flow:

- The system manages user input (e.g., searching for a movie or submitting a review) through React components.
- Data is sent via **JSON** payloads over HTTP requests to the Node.js server.
- The server validates the JWT token, processes the business logic in the controller layer, and stores or retrieves data from MongoDB collections.
- The response is sent back to the React client to update the UI instantly without a page reload via Redux Toolkit state management.

6. User Interface and Experience:

- The system emphasizes movie discovery through a homepage featuring New Releases, Top Rated, and Random movie carousels with genre-based filtering.
- **React-Bootstrap** is used to ensure the layout is consistent and responsive across all devices.

SYSTEM REQUIREMENTS

Minimum Hardware Requirements for Development

- **CPU:** Intel Core i5 or equivalent (Node.js and React dev servers require moderate processing power).
- **Memory (RAM):** 8 GB (4 GB is absolute minimum, but 8 GB is recommended for running MongoDB, Backend, and Frontend servers simultaneously).
- **Hard Disk:** 256 GB SSD (Solid State Drive is preferred for faster compilation and database reads).
- **System Type:** 64-bit Operating System.

Minimum Software Requirements for Development

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu 20.04+).
- **Runtime Environment:** Node.js (v18.0.0 or later).
- **Database:** MongoDB (Local installation or MongoDB Atlas Cloud).
- **Package Manager:** NPM (Node Package Manager) or Yarn.
- **Code Editor:** Visual Studio Code (VS Code) with extensions for ES7+ React/Redux, TypeScript, and Prettier.
- **API Testing:** Postman or Thunder Client.
- **Version Control:** Git and GitHub.
- **Browser:** Google Chrome (with React Developer Tools installed).

Minimum Hardware and Software Requirements for Running the Website (Client-Side)

On PCs (Desktops and Laptops)

- **CPU:** Intel Core i3 or equivalent.
- **Memory (RAM):** 4 GB or higher.
- **Internet Connection:** Broadband connection for loading dynamic content and real-time updates.
- **Web Browser:** Modern browsers supporting ES6 JavaScript features (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).

On Mobile Devices (Phones and Tablets)

- **CPU:** Quad-core processor (Snapdragon 600 series or equivalent) for smooth UI rendering.
- **Memory (RAM):** 3 GB or higher.
- **Operating System:** Android 10+ or iOS 14+.
- **Browser:** Chrome for Android, Safari, or Firefox Mobile.
- **Screen Resolution:** 720p HD or higher (The responsive design adjusts automatically to fit these screens).
-

SYSTEM DESIGN AND CODING

INTRODUCTION

Software design is a critical phase in the development of **Movie Streaming Application**, laying the architectural foundation for the entire application. It transcends basic coding by establishing a structured plan for how the MERN stack components (MongoDB, Express.js, React, Node.js) interact with TypeScript ensuring type safety across the full stack. Design is initiated after the system requirements (such as the need for Movie Management, JWT Authentication, and the Admin Dashboard) have been specified and analyzed. This step forms the core of the development phase and precedes the actual implementation of features. The primary goal of this phase is to create a scalable model of the forum that ensures seamless data flow between the client and server.

The central focus of the design for Movie Streaming Application is **User Experience (UX)** and **Data Integrity**. Through the design process, the quality of the final platform is ensured by translating user requirements—like the need for genre-filtered movie browsing and YouTube-embedded video streaming—into a structured framework of React Components and RESTful API Endpoints. A robust design minimizes the risk of bugs, ensures secure data handling (e.g., JWT authentication, bcrypt password encryption), and creates a system that is easy to maintain and expand in the future.

In this phase, detailed representations of data structures (Mongoose Schemas for Movie, User, Genre, Review, and MovieRequest), program structures (Redux Toolkit State Slices), and procedural steps (JWT Authentication Middleware) are defined and implemented. From both a

technical and project management perspective, this system design plays a pivotal role in guiding the project toward its successful deployment.

INPUT DESIGN

Input design is the process of converting user-oriented input into a computer-readable format. In **Movie Streaming Application**, input design focuses on the various forms and interactive elements users engage with to create content. As a key component of the overall system design, this was executed with meticulous attention to detail to ensure that students can easily ask questions, vote on polls, and customize their profiles without technical friction.

The primary objectives of input design in this project were:

1. **Efficiency:** To create input processes (like instant movie search or genre-based filtering) that are fast and minimize user effort.
2. **Accuracy:** To ensure high data integrity by implementing client-side validation (React with TypeScript) and server-side validation (Express.js/Mongoose) to prevent empty reviews or invalid movie data.
3. **User-Friendliness:** To ensure that complex actions, such as creating a movie review with a star rating, are intuitive and straightforward.

Input Data Considerations in Movie Streaming Application: When designing the input interfaces, the goal was to create a process that is logical and error-free.

- **Data Recording:** Capturing raw data from users via **React forms with TypeScript**. This includes text inputs for Movie Search Queries, star rating selectors for Reviews, and Multer file inputs for Movie Thumbnail and Cover Image uploads.
- **Data Conversion:** Converting user actions into JSON format. For example, when a user selects a Genre filter from a dropdown, it is converted into a query parameter sent to the Express.js backend API.
- **Data Verification:** Checking inputs before submission. For instance, the `reviewMovie` controller enforces one-review-per-user-per-movie validation, and the review form prevents submission if either the star rating or comment text is missing.
- **Interactive Input:** The system features real-time interactive inputs, such as the movie search bar which dynamically filters the catalog by title, and the star rating component which provides immediate visual feedback upon selection.

Specific Input Interfaces Designed:

- **Authentication Forms:** Secure inputs for Username, Email, and Password with server-side validation, bcrypt hashing, and JWT cookie-based session management.
- **Movie Management Form (Admin):** A comprehensive admin-only form for creating and editing movies, including fields for Name, Description, Release Year, Director, Genre (multi-select), Cast Members, Cover Image upload (via Multer), and YouTube Video ID for each episode.
- **Movie Review Form:** An interactive form where authenticated users can submit a 1–5 star rating and a text comment. The system enforces one review per user per movie and displays the average rating dynamically.

- **Movie Request Form:** A user-facing form where registered users can submit new movie title requests with a requester name, movie title, and optional description. Submitted requests are logged in MongoDB and appear in the Admin Dashboard for review.

OUTPUT DESIGN

Output design refers to the process of determining how the results of processing will be communicated to the users. In **Movie Streaming Application**, output design plays a crucial role in displaying movie catalogs, video content, and user reviews in a way that is visually rich and engaging for modern streaming audiences.

The outputs of the system are categorized as follows:

- **Dynamic Movie Feed Outputs:** The most common output type in this Single Page Application (SPA). This includes the **Movie Feed**, where data fetched from the MongoDB database is dynamically rendered as movie carousels (New Releases, Top Rated, Random Picks). When a user submits a review, the average rating updates instantly without a page reload.
- **Visual Outputs:** The **Movie Detail Page** is a key visual output. It displays the movie cover image, complete metadata (year, director, cast, genre badges), an aggregated star rating with review count, and an embedded YouTube video player for direct content streaming.
- **Operational Outputs:** System feedback such as "Review Submitted Successfully," "Movie Request Sent," or "Invalid Password" (via React Toastify notifications) helps users understand the result of their interactions in real time.

Key Output Design Principles Applied:

- **Responsiveness:** Using the Tailwind CSS v4 responsive utility classes, outputs are designed to adapt to any screen size. A movie catalog that displays as a multi-column grid on desktop automatically transforms into a scrollable vertical list on mobile devices.
- **Clarity:** Information is structured hierarchically. The Movie Title and cover image are prominent, followed by metadata (Year, Genre, Director), aggregated star rating, and finally the description, ensuring users can quickly evaluate a movie's relevance.
- **Navigation:** The layout includes a dynamic navigation bar (Output) that provides quick access to Home, Movie Search, User Profile, and Admin Dashboard, allowing users to navigate through the streaming platform effectively.

CONCLUSION:

In the software engineering process of **Movie Streaming Application**, system design was foundational for building a quality, reliable, and user-friendly movie streaming platform. The rigorous **Input Design** ensures that movie metadata, user reviews, and movie requests are captured accurately and validated before storage. Meanwhile, the thoughtful **Output Design** guarantees that streaming content is presented in a visually rich, hierarchically organized manner that encourages user discovery and engagement. By focusing on these design principles using the MERN stack, the system successfully meets the needs of modern streaming audiences, operating efficiently and remaining scalable for future enhancements such as AI recommendations and Redis caching.

DATA DIAGRAM

Table 1

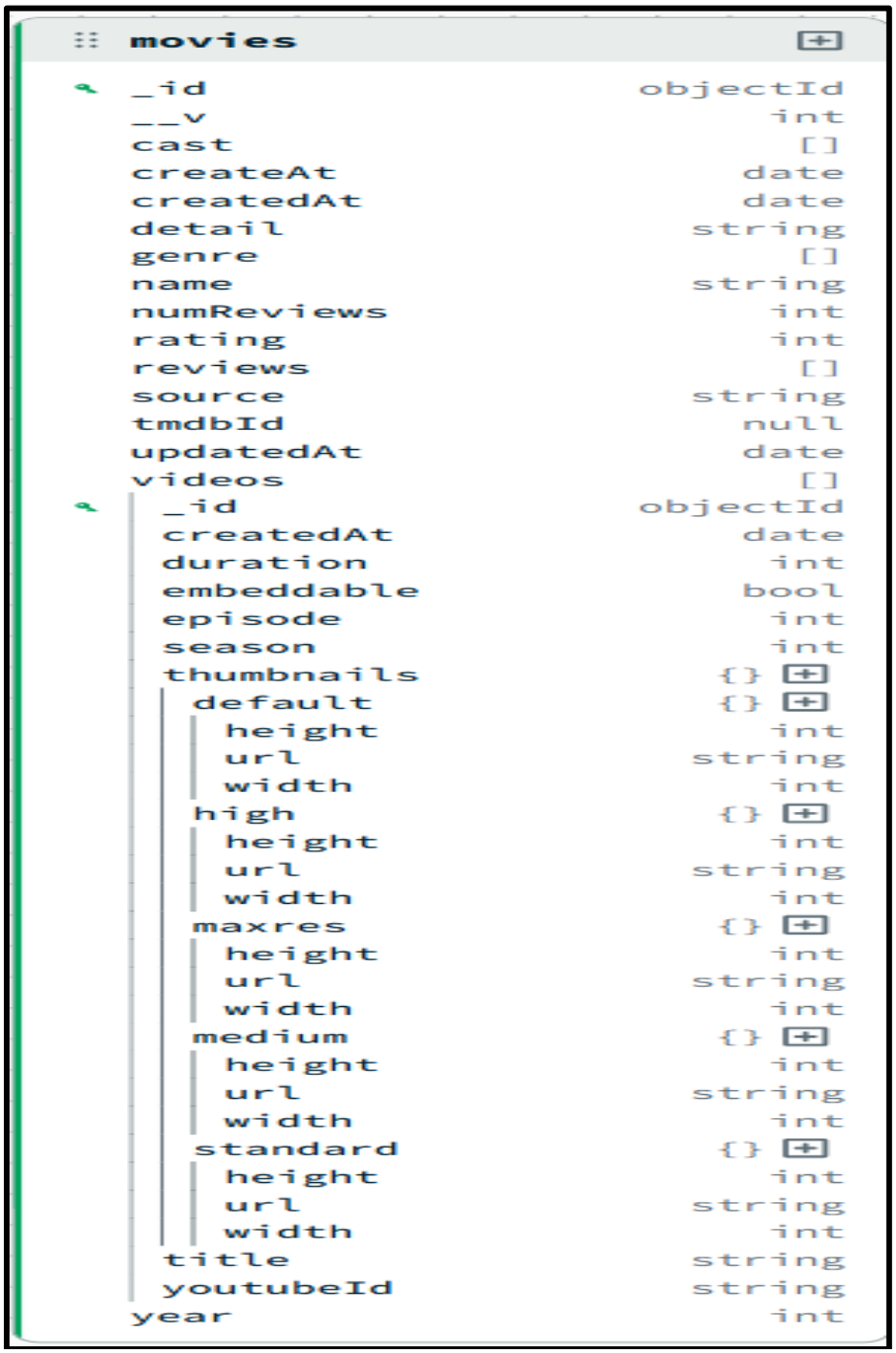
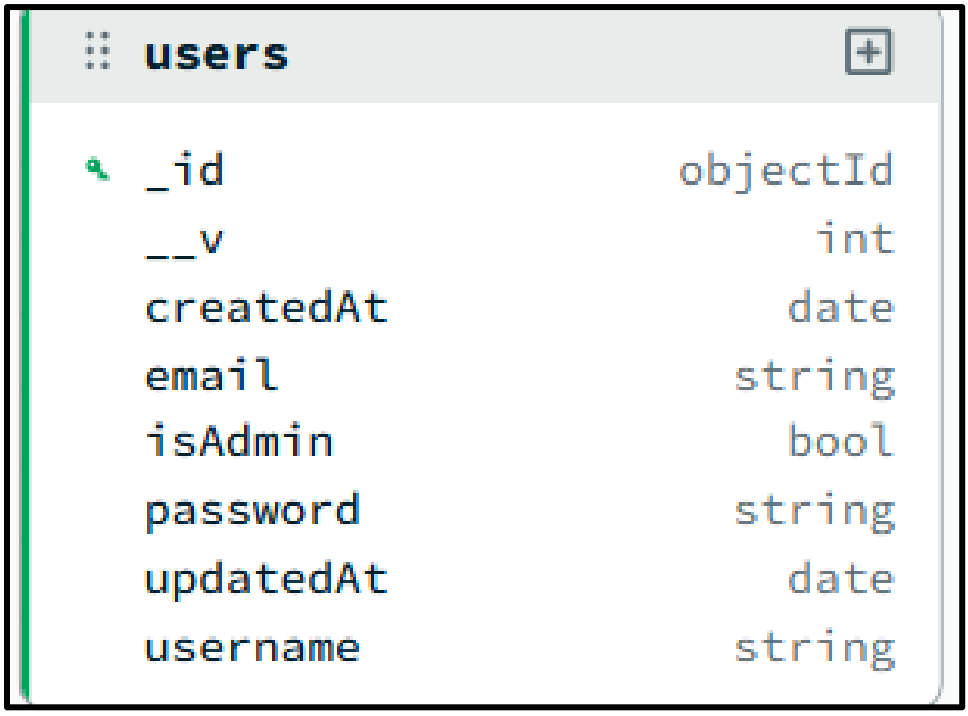


Table 2

Table 3



DATA FLOW DIAGRAM (DFD)

A Data Flow Diagram (DFD) is a graphical tool used to describe and analyze the flow of data through a system. It focuses on the data flowing into the system, the processes that transform the data, and the data flowing out to storage or external entities.

DFDs are of two types:

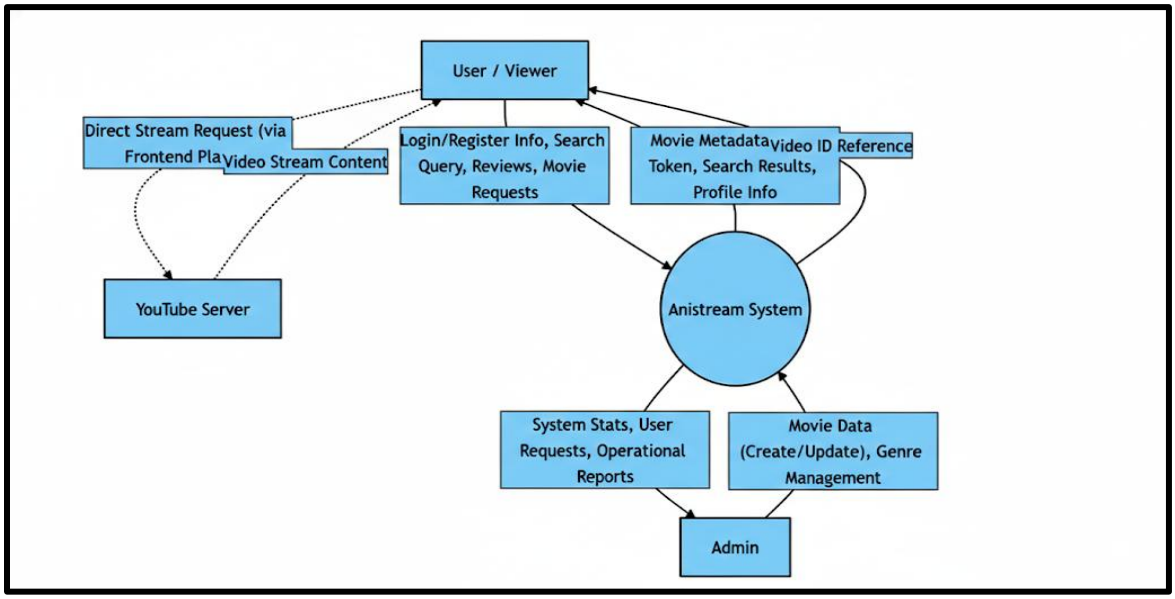
1. **Physical DFD:** The physical DFD is a model of the current system and is used to ensure that the current system has been clearly understood.
2. **Logical DFD:** The logical DFD is a model of the proposed system. It clearly shows the requirements on which the new system should be built, focusing on *what* happens, not *how* it happens.

Notation Used In DFD: There are four simple notations used to complete DFDs:

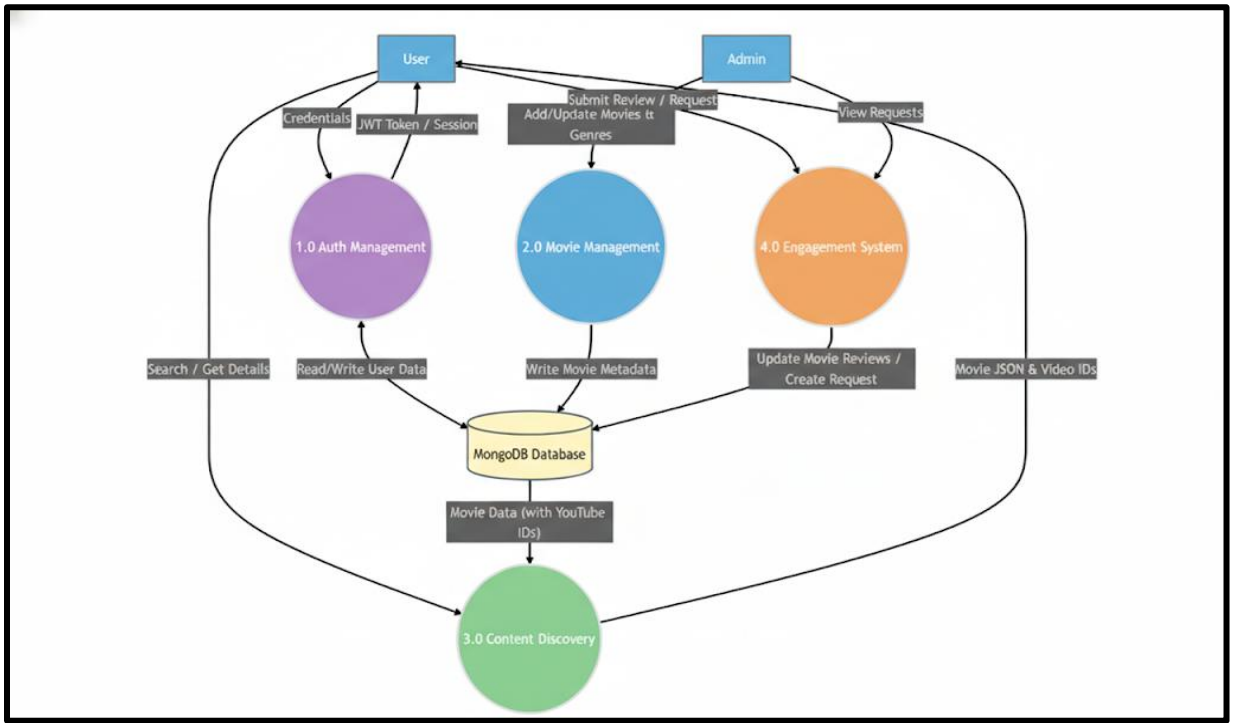
- **Data Flow (Arrow):** Represents the movement of data packets from one part of the system to another.
- **External Entity (Rectangle):** Represents sources (where data comes from) or destinations (where data goes) outside the system boundaries (e.g., Users, Admin).
- **Process (Circle/Rounded Rectangle):** Represents a function or logic that transforms incoming data into outgoing data (e.g., "Verify Login").
- **Data Store (Open Rectangle/Parallel Lines):** Represents repositories where data is stored for later use (e.g., Database Collections like Users, Topics).

DFD FOR THE SYSTEM

1. Level 0:



2. Level 1:



ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) expresses the overall logical structure of the database graphically. It illustrates the interrelationships between the entities in the Movie Streaming Application, providing a clear blueprint of how data is organized and connected within the MongoDB database using Mongoose schemas.

The components of the E-R Diagram are:

1. Entity: An entity is a real-world object or concept that is distinguishable from all other objects. In this project, the primary entities include User, Movie, Genre, Review, Video, and MovieRequest.
2. Relationship: It is an association among several entities. For instance, a "User" submits a "Review" for a "Movie," representing a relationship between these entities.
3. Weak Entity: A weak entity is an entity that cannot be uniquely identified by its own attributes alone and depends on a strong entity. In this system, a Review can be viewed conceptually as a weak entity because it is dependent on the existence of a specific Movie document.
4. Attributes: A property or characteristic of an entity. For example, a User entity has attributes like Username, Email, Password, isAdmin, and timestamps.

ENTITIES IN MOVIE STREAMING APPLICATION

Based on the Mongoose schemas designed for the system, the key entities and their attributes are:

1. User: Represents the registered viewers and administrators of the platform.

- Attributes: _id, username, email, password, isAdmin (Boolean), createdAt, updatedAt.

2. Movie: Represents the streaming content entries managed by administrators.

   - Attributes: _id, name, tmdbId, image (thumbnail), coverImage, year, detail, genre (ref: Genre[]), cast (String[]), director, reviews (Review[]), rating (avg), numReviews, videos (Video[]), source (enum: tmdb/youtube/other), createdAt, updatedAt.

3. Review (embedded subdocument in Movie): Represents ratings and text feedback submitted by users.

   - Attributes: _id, user (ref: User), rating (1-5), comment (String), name (String), createdAt, updatedAt.

4. Genre: Represents the classification categories (e.g., Action, Drama, Comedy) used to organize movies.

   - Attributes: _id, name (unique, maxlength: 32).

5. Video (embedded subdocument in Movie): Represents individual episodes or video segments linked via YouTube.

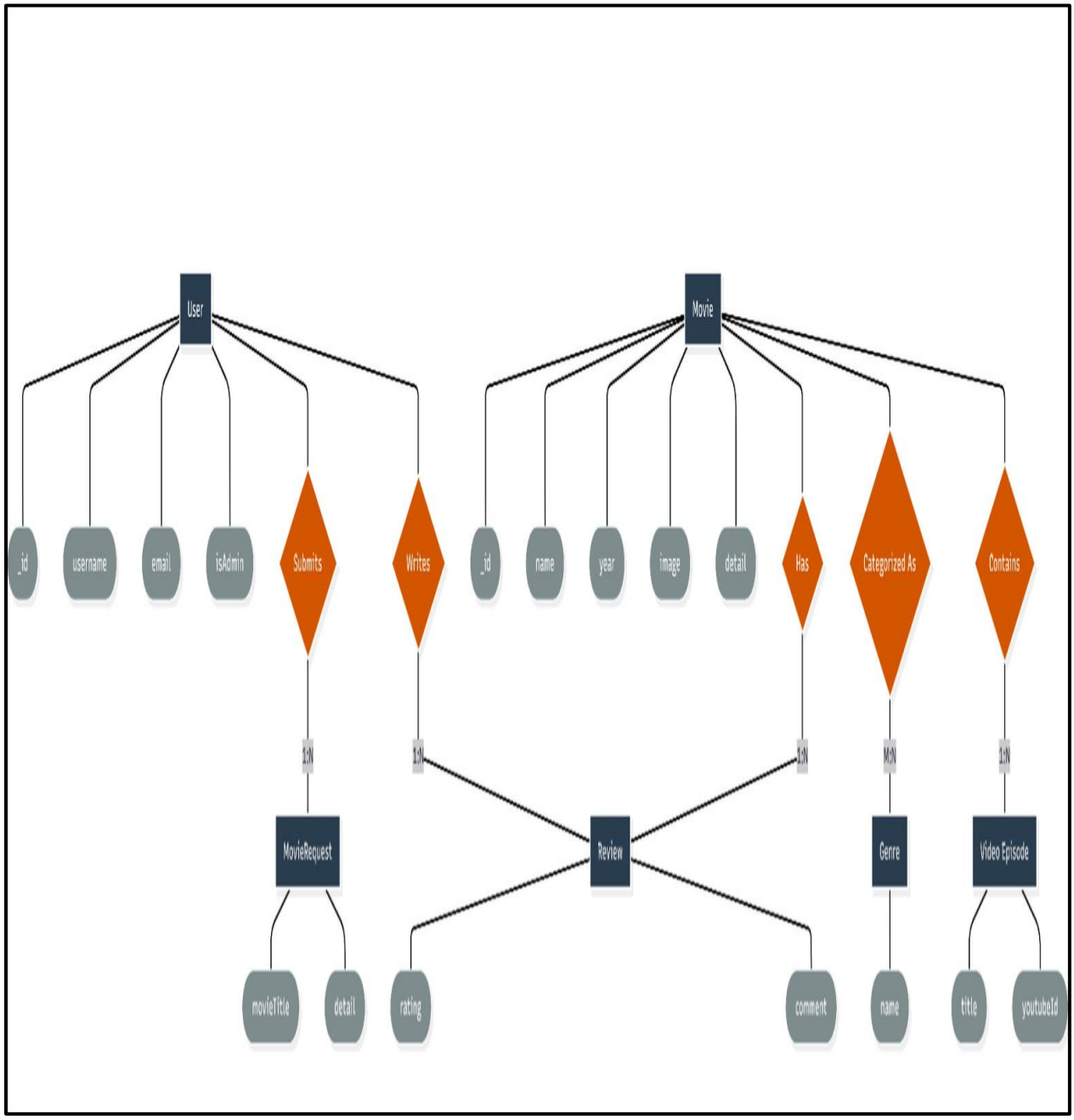
   - Attributes: _id, title, youtubeId, season, episode, duration (seconds), publishedAt, thumbnails, embeddable (Boolean), createdAt.

RELATIONSHIPS

The system enforces the following relationships to ensure data integrity and connectivity:

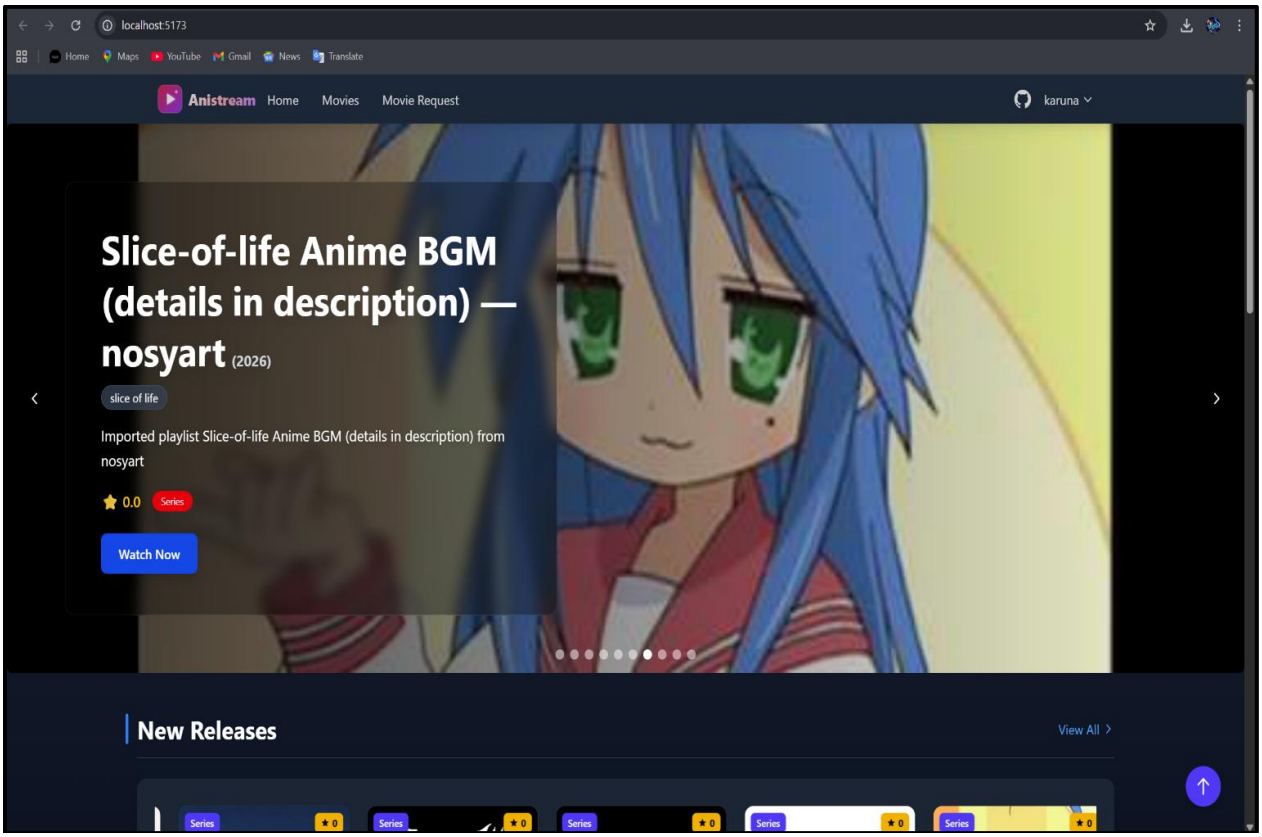
1. User — Review (1:N): A single User can submit reviews for multiple Movies, with one review enforced per user per movie.
2. Admin User — Movie (1:N): An administrator can create and manage multiple Movies in the streaming catalog.
3. Movie — Review (1:N): A single Movie document can contain many embedded Reviews from different users.
4. Genre — Movie (1:N): A Genre acts as a classification container for multiple Movies (referenced via ObjectId array).
5. Movie — Video (1:N): A single Movie document can embed multiple Video subdocuments representing different episodes or streaming segments.
6. User — MovieRequest (1:N): A User can submit multiple MovieRequests, and each request references the submitting user via ObjectId.

ENTITY RELATIONSHIP DIAGRAM



INPUT FORMS, PAGES AND CODING

1. Home Page (Movie Listing):



Code:

File: frontend/src/pages/Home.tsx-

```
import BackToTopButton from "../components/BackToTopButton";
import Footer from "../components/Footer";
import Header from "./Movies/Header";
import MoviesContainerPage from "./Movies/MoviesContainerPage";

const Home = () => {
   return (
      <div className="bg-gray-900 min-h-screen text-white">
         <div classNAme="">
            <Header />
         </div>
         <MoviesContainerPage />
         <BackToTopButton />
         <Footer />
```

```
      </div>
   );
};

export default Home;
```

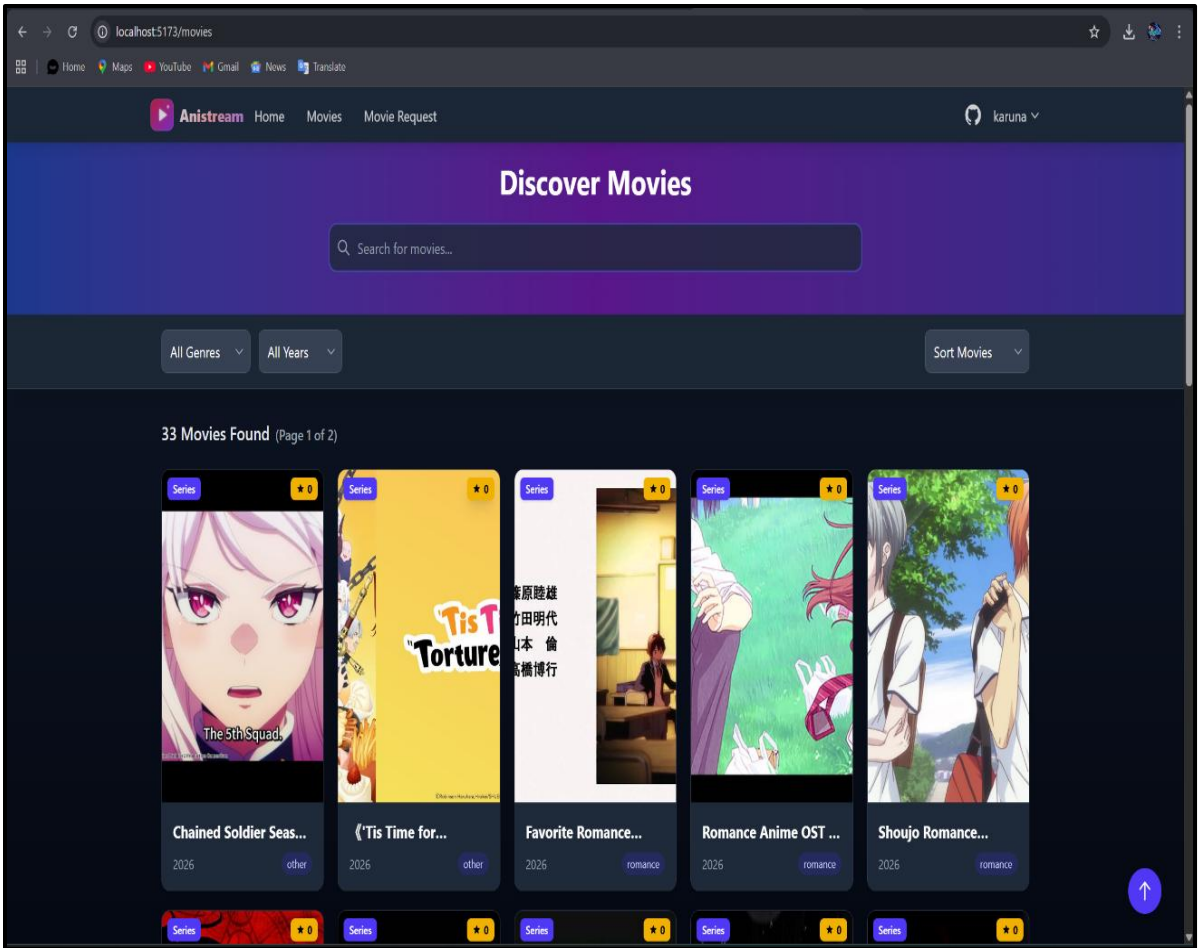
File: frontend/src/pages/Movies/Header.tsx-

```
import { Link } from "react-router-dom";
import HeroSlider from "../../components/HeroSlider";
import { useGetNewMoviesQuery } from "../../redux/api/movies";

const Header = () => {
   const { data: newMovies } = useGetNewMoviesQuery();

   return (
      <div className="flex flex-col mt-[2rem] ml-[2rem] md:flex-row justify-between items-center md:items-start gap-4">
         <nav className="w-full md:w-[12rem] ml-0 md:ml-2 mb-4 md:mb-0">
            <Link
               to="/"
               className="transition duration-300 ease-in-out hover:bg-teal-200 block p-2 rounded mb-1 md:mb-2 text-lg font-bold"
            >
               Home
            </Link>
            <Link
               to="/movies"
               className="transition duration-300 ease-in-out hover:bg-teal-200 block p-2 rounded mb-1 md:mb-2 text-lg font-bold"
            >
               Browse Movies
            </Link>
         </nav>

         <div className="w-full md:w-[80%] mr-0 md:mr-2">
            <HeroSlider movies={newMovies || []} />
         </div>
      </div>
   );
};

export default Header;
```

File: frontend/src/components/HeroSlider.tsx-

```
import Slider from "react-slick";
import "slick-carousel/slick/slick.css";
import "slick-carousel/slick/slick-theme.css";
import { Movie } from "../types/movieTypes";

interface HeroSliderProps {
  movies: Movie[];
}

const HeroSlider: React.FC<HeroSliderProps> = ({ movies }) => {
  const settings = {
    dots: true,
    infinite: true,
    speed: 500,
    slidesToShow: 1,
    slidesToScroll: 1,
    autoplay: true,
    autoplaySpeed: 3000,
  };

  return (
    <Slider {...settings}>
      {movies.map((movie) => (
        <div key={movie._id} className="relative h-[24rem]">
          <img
            src={movie.image}
            alt={movie.name}
            className="w-full h-full object-cover rounded-lg"
          />
          <div className="absolute bottom-0 left-0 w-full bg-gradient-to-t from-black to-transparent p-4">
            <h2 className="text-2xl font-bold text-white">{movie.name}</h2>
            <p className="text-gray-300">{movie.detail?.substring(0, 100)}...</p>
          </div>
        </div>
      ))}
    </Slider>
  );
};

export default HeroSlider;
```

2. Movie section:



Code:

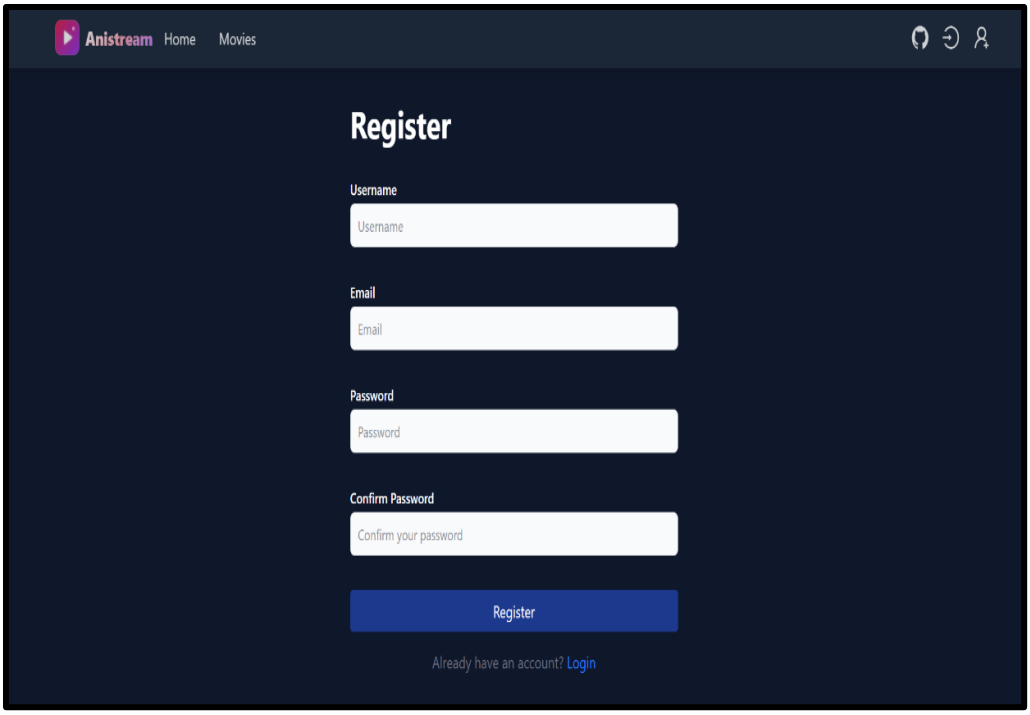
```
import { useGetAllMoviesQuery } from "../../redux/api/movies";
import { useFetchGenresQuery } from "../../redux/api/genre";
import {
  useGetNewMoviesQuery,
  useGetTopMoviesQuery,
  useGetRandomMoviesQuery,
} from "../../redux/api/movies";
import MovieCard from "./MovieCard";
import { useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";
import banner from "../../assets/banner.jpg";
import {
  setMoviesFilter,
  setFilteredMovies,
  setMovieYears,
```

```
  setUniqueYears,
} from "../../redux/features/movies/moviesSlice";

const AllMovies = () => {
  const dispatch = useDispatch();
  const { data } = useGetAllMoviesQuery();
  const { data: genres } = useFetchGenresQuery();
  const { data: newMovies } = useGetNewMoviesQuery();
  const { data: topMovies } = useGetTopMoviesQuery();
  const { data: randomMovies } = useGetRandomMoviesQuery();

  const { moviesFilter, filteredMovies } = useSelector((state: any) => state.movies);

  const movieYears = data?.map((movie: any) => movie.year);
  const uniqueYears = Array.from(new Set(movieYears));

  useEffect(() => {
    dispatch(setFilteredMovies(data || []));
    dispatch(setMovieYears(movieYears));
    dispatch(setUniqueYears(uniqueYears));
  }, [data, dispatch]);

  const handleSearchChange = (e: any) => {
    dispatch(setMoviesFilter({ searchTerm: e.target.value }));

    const filteredMovies = data.filter((movie: any) =>
      movie.name.toLowerCase().includes(e.target.value.toLowerCase())
    );

    dispatch(setFilteredMovies(filteredMovies));
  };

  const handleGenreClick = (genreId: string) => {
    const filterByGenre = data.filter((movie: any) => movie.genre === genreId);
    dispatch(setFilteredMovies(filterByGenre));
  };

  const handleYearChange = (year: any) => {
    const filterByYear = data.filter((movie: any) => movie.year === +year);
    dispatch(setFilteredMovies(filterByYear));
  };

  const handleSortChange = (sortOption: string) => {
    switch (sortOption) {
      case "new":
        dispatch(setFilteredMovies(newMovies));
```

```
            break;
        case "top":
            dispatch(setFilteredMovies(topMovies));
            break;
        case "random":
            dispatch(setFilteredMovies(randomMovies));
            break;
        default:
            dispatch(setFilteredMovies(data));
            break;
    }
  };

  return (
    <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-4 -translate-y-[5rem]">
        <>
          <section>
            <div
              className="relative h-[50rem] w-screen mb-10 flex items-center justify-center bg-cover"
              style={{ backgroundImage: `url(${banner})` }}
            >
              <div className="absolute inset-0 bg-gradient-to-b from-gray-800 to-black opacity-60"></div>

              <div className="relative z-10 text-center text-white mt-[10rem]">
                <h1 className="text-8xl font-bold mb-4">The Movies Hub</h1>
                <p className="text-2xl">
                  Cinematic Odyssey: Unveiling the Magic of Movies
                </p>
              </div>

              <section className="absolute -bottom-[5rem]">
                <input
                  type="text"
                  className="w-[100%] h-[5rem] border px-10 outline-none rounded"
                  placeholder="Search Movie"
                  value={moviesFilter.searchTerm}
                  onChange={handleSearchChange}
                />
                <section className="sorts-container mt-[2rem] ml-[10rem]  w-[30rem]">
                  <select
                    className="border p-2 rounded text-black"
                    onChange={(e) => handleGenreClick(e.target.value)}
                  >
```

```
                        <option value="">Genres</option>
                        {genres?.map((genre: any) => (
                          <option key={genre._id} value={genre._id}>
                            {genre.name}
                          </option>
                        ))}
                      </select>
                      <select
                        className="border p-2 rounded ml-4 text-black"
                        onChange={(e) => handleYearChange(e.target.value)}
                      >
                        <option value="">Year</option>
                        {uniqueYears.map((year: any) => (
                          <option key={year} value={year}>
                            {year}
                          </option>
                        ))}
                      </select>

                      <select
                        className="border p-2 rounded ml-4 text-black"
                        onChange={(e) => handleSortChange(e.target.value)}
                      >
                        <option value="">Sort By</option>
                        <option value="new">New Movies</option>
                        <option value="top">Top Rated</option>
                        <option value="random">Random</option>
                      </select>
                    </section>
                  </section>
                </div>

                <section className="mt-[10rem] w-screen flex justify-center items-center flex-
wrap">
                  {filteredMovies?.map((movie: any) => (
                    <MovieCard key={movie._id} movie={movie} />
                  ))}
                </section>
              </section>
            </>
          </div>
        );
};

export default AllMovies;
```

3. Register Page:



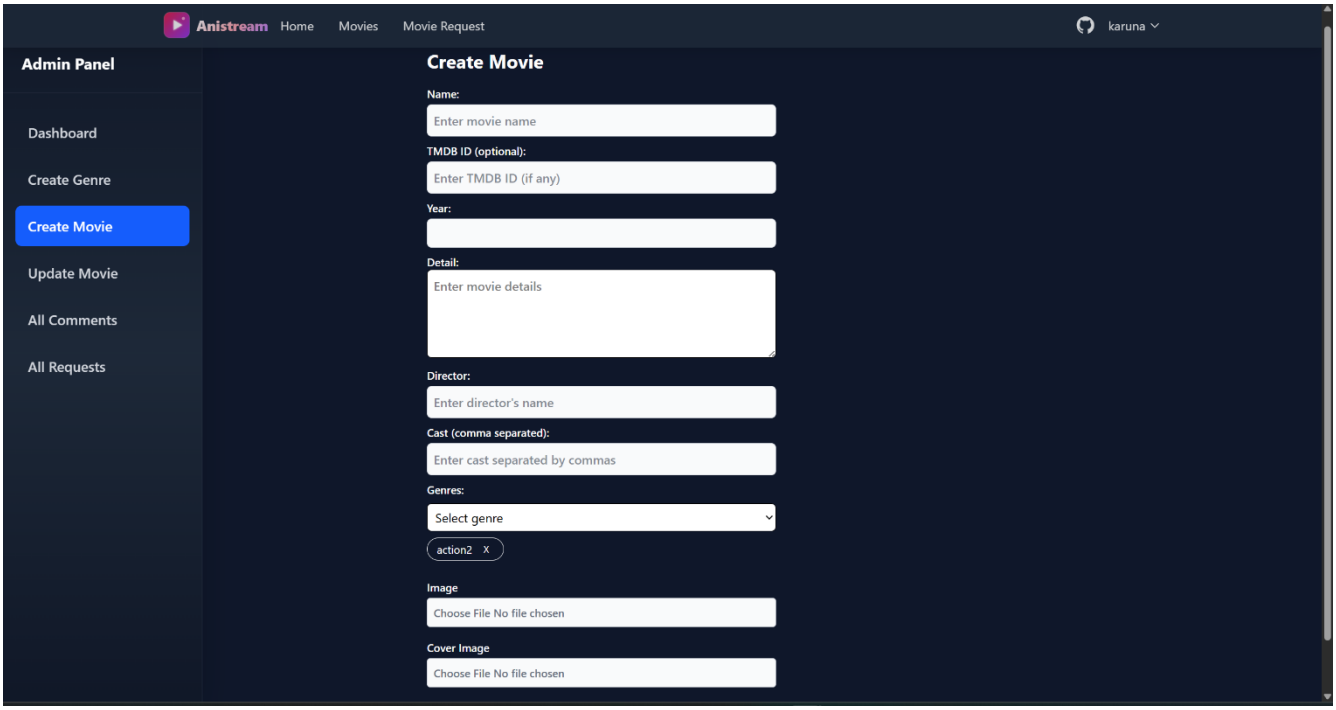
Code:

```
import { useState, useEffect } from "react";
import { useDispatch, useSelector } from "react-
redux";
import { useNavigate } from "react-router-dom";
import { Form, Button, Row, Col, Card,
InputGroup } from "react-bootstrap";
import { register, resetRegister } from
"../redux/slices/authSlice";
const Register = () => {
  // Local State for Form Inputs
  const [formData, setFormData] = useState({
    firstName: "", lastName: "", username: "",
email: "", password: ""
  });
  const navigate = useNavigate();
  const dispatch = useDispatch();
  // Redux State: Auth Status & Registration
Feedback
  const { isLoggedIn } = useSelector((state) =>
state.auth);
```

```
 const { isLoading, message, isError } = useSelector((state) => state.auth.register);
 // Effect: Redirect if already logged in
 useEffect(() => {
  if (isLoggedIn) navigate("/");
 }, [isLoggedIn, navigate]);

 // Effect: Clear previous registration messages
 useEffect(() => {
  dispatch(resetRegister());
 }, [dispatch]);
 const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
 };
 const handleSubmit = (e) => {
  e.preventDefault();
  if (Object.values(formData).every(field => field !== "")) {
   dispatch(register(formData));
  }
 };
 return (
  <Row className="justify-content-center align-items-center" style={{ minHeight: "80vh" }}>
   <Col lg={6} md={8}>
    <Card className="shadow-sm">
     <Card.Body className="p-4">
      <h3 className="text-center mb-4">Create Account</h3>

      {/* Feedback Message */}
      {message && (
       <div className={`alert ${isError ? "alert-danger" : "alert-success"}`}>
        {message}
       </div>
      )}

      <Form onSubmit={handleSubmit}>
       <Row>
        <Col md={6}>
         <Form.Group className="mb-3">
          <Form.Label>First Name</Form.Label>
          <Form.Control name="firstName"
```

```
onChange={handleChange} required />
              </Form.Group>
            </Col>
            <Col md={6}>
              <Form.Group className="mb-3">
                <Form.Label>Last
Name</Form.Label>
                <Form.Control      name="lastName"
onChange={handleChange} required />
              </Form.Group>
            </Col>
          </Row>
          <Form.Group className="mb-3">

<Form.Label>Username</Form.Label>
            <Form.Control        name="username"
onChange={handleChange} required />
          </Form.Group>
          <Form.Group className="mb-3">
            <Form.Label>Email
Address</Form.Label>
            <Form.Control              type="email"
name="email"         onChange={handleChange}
required />
          </Form.Group>
          <Form.Group className="mb-4">
            <Form.Label>Password</Form.Label>
            <Form.Control         type="password"
name="password"    onChange={handleChange}
required />
          </Form.Group>
          <Button  type="submit"  className="w-
100" disabled={isLoading}>
            {isLoading     ?     "Registering..."     :
"Register"}
          </Button>
        </Form>
      </Card.Body>
    </Card>
  </Col>
 </Row>
 );
};

export default Register;
```

4. Create Movie (Admin)



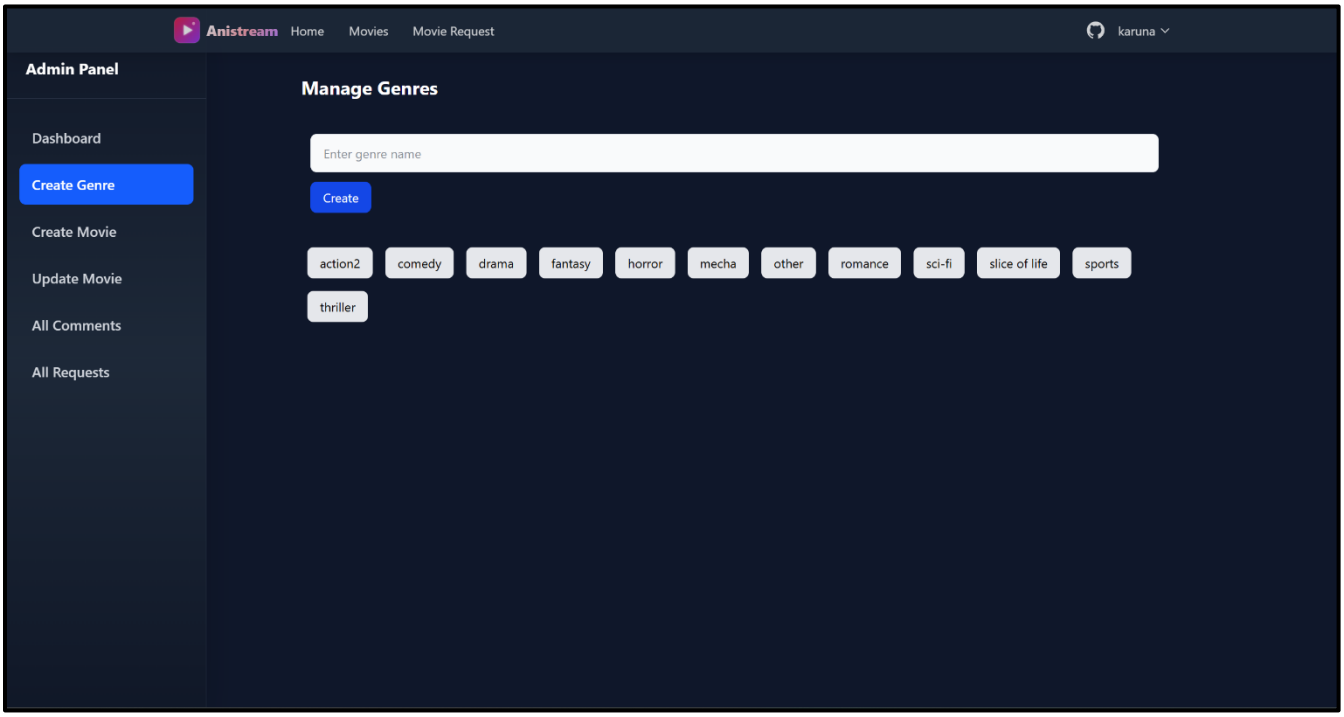
Code:

```
import { useState, useEffect } from "react";
import { useDispatch, useSelector } from "react-
redux";
import { Container, Row, Col, Form, Button }
from "react-bootstrap";
import Select from "react-select";
import CreatableSelect from "react-
select/creatable";
import { addTopic, getSpaces, resetNewTopic }
from "../redux/slices/topicSlice";
import CreatePoll from
"../components/Topic/Poll/CreatePoll";

const NewTopic = () => {
  const dispatch = useDispatch();

  // Local State
  const [title, setTitle] = useState("");
```

```
  const [content, setContent] = useState("");
  const [selectedSpace, setSelectedSpace] = useState(null);
  const [selectedTags, setSelectedTags] = useState([]);
  const [showPoll, setShowPoll] = useState(false);
  const [pollData, setPollData] = useState(null);

  // Redux State
  const { spaces } = useSelector((state) => state.topic);
  const { isLoading, message, isSuccess } = useSelector((state) => state.topic.addTopic);

  // Fetch Spaces on Mount
  useEffect(() => {
    dispatch(getSpaces());
    dispatch(resetNewTopic());
  }, [dispatch]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (title && content && selectedSpace) {
      dispatch(addTopic({
        title,
        content,
        selectedSpace: selectedSpace.value,
        selectedTags,
        poll: showPoll ? pollData : null // Attach Poll Data if active
      }));
    }
  };

  return (
    <Container className="py-5">
      <Row className="justify-content-center">
        <Col lg={8}>
          <h3 className="mb-4">Create New Topic</h3>

          {message && <div className={`alert ${isSuccess ? "alert-success" : "alert-danger"}`}>{message}</div>}
          <Form onSubmit={handleSubmit}>
            <Form.Group className="mb-3">
```

```
          <Form.Label>Topic Title</Form.Label>
          <Form.Control value={title}
onChange={(e) => setTitle(e.target.value)}
required />
        </Form.Group>
        <Form.Group className="mb-3">
          <Form.Label>Space</Form.Label>
          <Select
            options={spaces?.map(s => ({ value:
s.name, label: s.name }))}
            onChange={setSelectedSpace}
          />
        </Form.Group>
        <Form.Group className="mb-3">
          <Form.Label>Content</Form.Label>
          <Form.Control as="textarea" rows={5}
value={content} onChange={(e) =>
setContent(e.target.value)} required />
        </Form.Group>
        <Form.Group className="mb-3">
          <Form.Label>Tags</Form.Label>
          <CreatableSelect isMulti
onChange={setSelectedTags} />
        </Form.Group>
        {/* Poll Toggle Section */}
        <div className="mb-4">
          <Button variant="outline-primary"
size="sm" onClick={() =>
setShowPoll(!showPoll)}>
            {showPoll ? "Remove Poll" : "Add
Poll"}
          </Button>
          {showPoll && <CreatePoll
onPollChange={setPollData} />}
        </div>
        <Button type="submit"
variant="primary" size="lg"
disabled={isLoading}>
          {isLoading ? "Posting..." : "Create
Topic"} </Button>
       </Form>
      </Col>
    </Row></Container>
  );
};
export default NewTopic;
```

5. Genre Management (Admin)



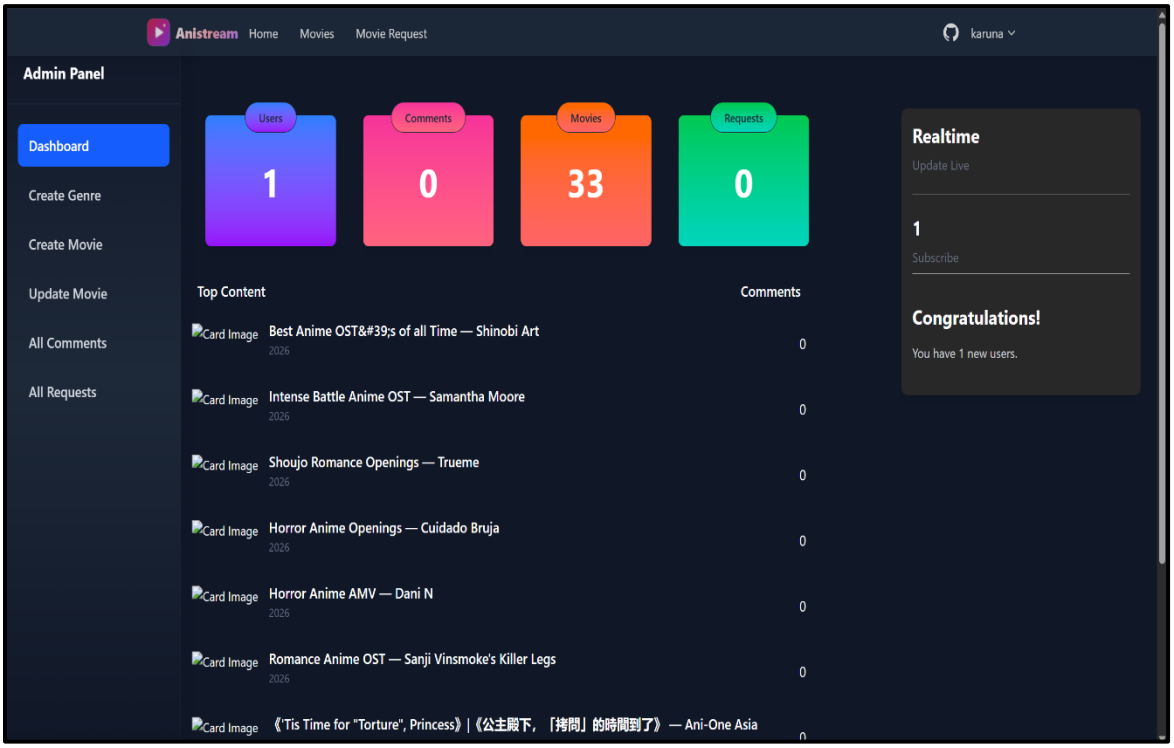
Code:

```
import { useEffect, useState } from "react";
import { useDispatch, useSelector } from "react-
redux";
import { Container, Table, Button, Form, Row,
Col } from "react-bootstrap";
import { getAllTags, deleteTag, createTag }
from "../../redux/slices/adminSlice";
import { MdDelete } from "react-icons/md";

const AdminTags = () => {
  const dispatch = useDispatch();
  const { tags, isLoading } = useSelector((state)
=> state.admin);
  const [newTag, setNewTag] = useState("");

  useEffect(() => {
    dispatch(getAllTags());
  }, [dispatch]);
```

```
  const handleAddTag = (e) => {
    e.preventDefault();
    if (newTag.trim()) {
      dispatch(createTag({ name: newTag }));
      setNewTag("");
    }
  };

  return (
    <Container className="py-4">
      <div  className="d-flex  justify-content-
between align-items-center mb-4">
        <h3>Manage Tags</h3>
        <Form  onSubmit={handleAddTag}
className="d-flex gap-2">
          <Form.Control
            placeholder="New Tag Name"
            value={newTag}
            onChange={(e)  =>
setNewTag(e.target.value)}
          />
          <Button  type="submit"
variant="primary">Add</Button>
        </Form>
      </div>

      <Table  striped  hover  responsive
className="shadow-sm">
        <thead className="bg-light">
          <tr>
            <th>#</th>
            <th>Tag Name</th>
            <th>Usage Count</th>
            <th>Created By</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody>
          {tags?.map((tag, index) => (
            <tr key={tag._id}>
              <td>{index + 1}</td>
              <td><span  className="badge  bg-
secondary">{tag.name}</span></td>
              <td>{tag.topicsCount || 0}</td>
              <td>{tag.creator || "Admin"}</td>
              <td>
```

```
              <Button
                variant="outline-danger"
                size="sm"
                onClick={()                              =>
dispatch(deleteTag(tag._id))}
              >
                <MdDelete /> Delete
              </Button>
            </td>
          </tr>
        ))}
      </tbody>
    </Table>
  </Container>
 );
};

export default AdminTags;
```

6. Admin panel dashboard



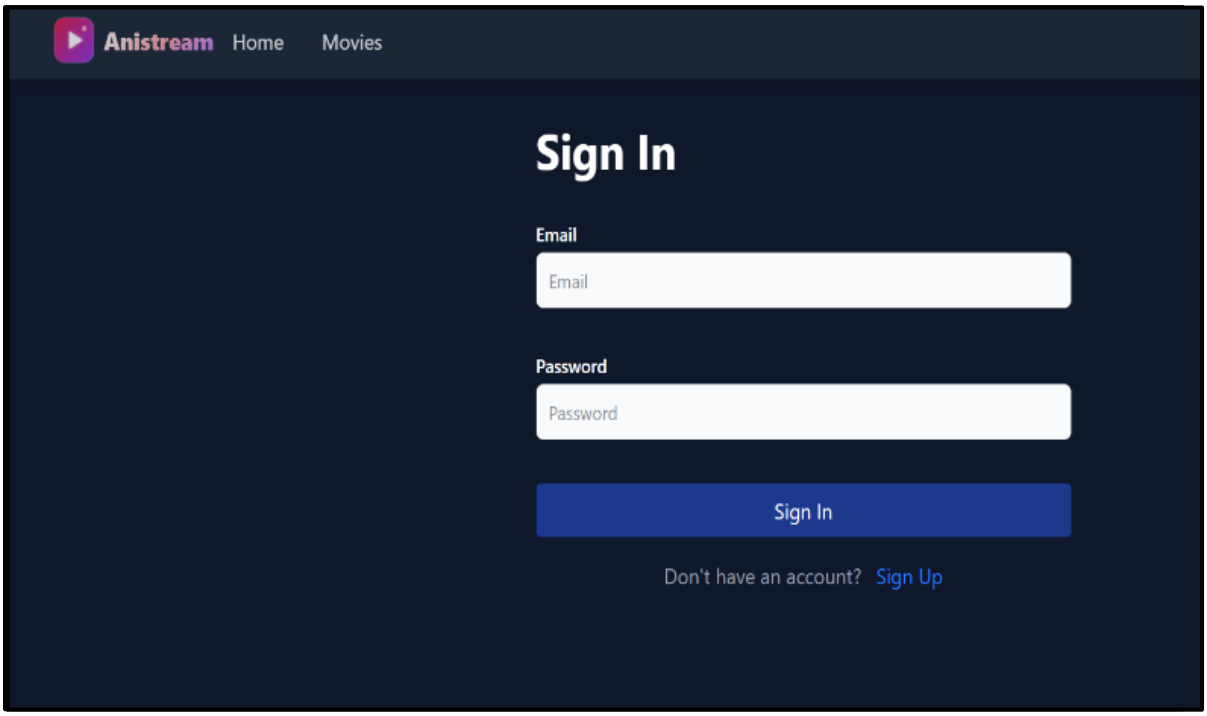
Code:

```
import Sidebar from "./Sidebar/Sidebar";
import Main from "./Main/Main";

const AdminDashBoard = () => {
 return (
  <section className="xl:ml-[4rem] md:ml-[0rem]">
   <div className="w-full flex">
    <Sidebar />
    <Main />
   </div>
  </section>
 );
};

export default AdminDashBoard;
import { Link } from "react-router-dom";
import { FaList, FaPlus, FaUsers } from "react-icons/fa";
```

```
const Sidebar = () => {
   return (
      <div className="-translate-y-10 flex h-screen fixed mt-10 border-r-2 border-[#242424]">
         <aside className="text-white w-64 flex-shrink-0">
            <ul className="py-4">
               <li className="text-lg  font-bold px-6 py-2">
                  <Link
                     to="/admin/movies/dashboard"
                     className="block p-3 rounded hover:bg-gray-800 transition duration-200"
                  >
                     Dashboard
                  </Link>
               </li>
               <li className="text-lg px-6 py-2">
                  <Link
                     to="/admin/movies/create"
                     className="flex items-center p-3 rounded hover:bg-gray-800 transition duration-200"
                  >
                     <FaPlus className="mr-2" /> Create Movie
                  </Link>
               </li>
               <li className="text-lg px-6 py-2">
                  <Link
                     to="/admin/movies/genre"
                     className="flex items-center p-3 rounded hover:bg-gray-800 transition duration-200"
                  >
                     <FaList className="mr-2" /> Create Genre
                  </Link>
               </li>
               <li className="text-lg px-6 py-2">
                  <Link
                     to="/admin/movies-list"
                     className="flex items-center p-3 rounded hover:bg-gray-800 transition duration-200"
                  >
                     <FaList className="mr-2" /> Update Movie
```

```
                </Link>
              </li>
              <li className="text-lg px-6 py-2">
                <Link
                  to="/admin/movies/comments"
                  className="flex items-center p-3 rounded hover:bg-gray-800 transition
duration-200"
                >
                  <FaUsers className="mr-2" /> Comments
                </Link>
              </li>
            </ul>
          </aside>
        </div>
    );
};

export default Sidebar;
```

7. Sign in page



Code:

```
import { useState, useEffect } from "react";
import { Link, useLocation, useNavigate } from "react-router-dom";
import { useDispatch, useSelector } from "react-redux";
import { useLoginMutation } from "../../redux/api/users";
import { setCredentials } from "../../redux/features/auth/authSlice";
import { toast } from "react-toastify";
import Loader from "../../components/Loader";
import loginImage from "../../assets/login.png";

const Login = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const dispatch = useDispatch();
  const navigate = useNavigate();

  const [login, { isLoading }] = useLoginMutation();
```

```
const { userInfo } = useSelector((state: any) => state.auth);

const { search } = useLocation();
const sp = new URLSearchParams(search);
const redirect = sp.get("redirect") || "/";

useEffect(() => {
  if (userInfo) {
    navigate(redirect);
  }
}, [navigate, redirect, userInfo]);

const submitHandler = async (e: React.FormEvent) => {
  e.preventDefault();

  try {
    const res = await login({ email, password }).unwrap();
    dispatch(setCredentials({ ...res }));
    navigate(redirect);
    toast.success("Login Successfully");
  } catch (err: any) {
    toast.error(err?.data?.message || err.error);
  }
};

return (
  <section className="pl-[10rem] flex flex-wrap">
    <div className="mr-[4rem] mt-[5rem]">
      <h1 className="text-2xl font-semibold mb-4 text-white">Sign In</h1>

      <form onSubmit={submitHandler} className="container w-[30rem]">
        <div className="my-[2rem]">
          <label
            htmlFor="email"
            className="block text-sm font-medium text-white"
          >
            Email Address
          </label>
          <input
```

```
            type="email"
            id="email"
            className="mt-1 p-2 border rounded w-full bg-gray-800 text-white border-gray-600 focus:outline-none focus:border-teal-500"
            placeholder="Enter email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
          />
        </div>

        <div className="mb-4">
          <label
            htmlFor="password"
            className="block text-sm font-medium text-white"
          >
            Password
          </label>
          <input
            type="password"
            id="password"
            className="mt-1 p-2 border rounded w-full bg-gray-800 text-white border-gray-600 focus:outline-none focus:border-teal-500"
            placeholder="Enter password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
          />
        </div>

        <button
          disabled={isLoading}
          type="submit"
          className="bg-teal-500 text-white px-4 py-2 rounded cursor-pointer my-[1rem]"
        >
          {isLoading ? "Signing In..." : "Sign In"}
        </button>

        {isLoading && <Loader />}
      </form>

      <div className="mt-4">
```

```
          <p className="text-white">
            New Customer?{" "}
            <Link
              to={redirect ? `/register?redirect=${redirect}` : "/register"}
              className="text-teal-500 hover:underline"
            >
              Register
            </Link>
          </p>
        </div>
      </div>

      <div className="hidden md:block md:w-[55%] bg-gray-900 h-[100vh]">
        <img
          src={loginImage}
          alt=""
          className="h-[65rem] w-full object-cover rounded-lg"
        />
      </div>
    </section>
  );
};

export default Login;
```

LIMITATIONS

1. Reliance on Third-Party Hosting (YouTube)

- The Limitation: Your application does not actually host the video files (like MP4s) on its own server. Instead, it embeds videos using youtubeId.
- Why it matters: If a video is deleted or made private on YouTube, it will stop working on your site immediately. You don't have full control over the content availability.

2. No "Smart" Personalization

- The Limitation: Currently, your "Recommendations" are likely based on general categories like "Top Rated" or "Newest".
- Why it matters: A commercial platform like Netflix uses AI to say, "Because you watched *Action Movie A*, you will like *Action Movie B*." Your system treats every user mostly the same rather than learning their specific tastes.

3. Limited Search Capabilities

- The Limitation: Your search functionality filters movies by name matching what the user types.
- Why it matters: It works great for a few hundred movies. However, if the database grew to 100,000 movies, this simple search method would become slow. It also likely doesn't handle typos well (e.g., if a user types "Avengrs" instead of "Avengers", it might return nothing).

4. Requires Constant Internet Connection

- The Limitation: There is no "Offline Mode" or "Download" feature.
- Why it matters: Users cannot watch movies while traveling (like on a plane) or in areas with poor internet. The app is purely a "streaming" service, not a "downloading" service.

5. Manual Content Management

- The Limitation: Adding movies is a manual process where the Admin has to fill out a form (Name, Image, YouTube ID).
- Why it matters: This is fine for a college project, but for a real-world app, you would need an automated system to "crawl" the web and add thousands of movies automatically without a human typing in every detail.

6. No Adaptive Streaming Quality

- The Limitation: Professional streaming services automatically switch between 480p, 720p, and 1080p based on the user's internet speed.
- Why it matters: Your player relies on YouTube's player to handle this. You haven't built a custom video player that handles "buffering" or quality switching on your own backend.

FUTURE SCOPE OF GAURAV'S ANISTREAM

The current implementation of the Movie Streaming Application serves as a robust full-stack streaming foundation. However, there is significant potential for expansion and enhancement in future iterations:

1. **User Watchlist & Favorites:** Integrating **Watchlist & Favorites** functionality to save movies for later viewing, track watch history, and build curated lists. This would significantly enhance user engagement and retention on the platform.
2. **AI-Powered Movie Recommendations:** Integrating an AI-driven recommendation engine using **collaborative filtering** or TensorFlow.js to analyze a user's viewing history, genre preferences, and past ratings to suggest personalized movies, similar to modern streaming platform recommendation algorithms.
3. **Redis Caching Layer:** Adding a Redis caching layer to the backend to cache frequently accessed data such as top-rated movies, genre lists, and new releases. This would dramatically reduce MongoDB query load and improve API response times for high-traffic scenarios.
4. **Production Deployment & Mobile App:** Expanding the "Spaces" feature to include a dedicated **Document Repository**. This would leverage the existing RESTful API to deliver a seamless cross-platform experience with push notifications for new movie releases and personalized alerts.
5. **External API Integration & Content Crawling:** Integrating the TMDB (The Movie Database) API to automatically populate movie metadata, posters, and cast information. Additionally, implementing data crawling scripts (using Puppeteer or Cheerio) to

aggregate publicly available streaming content and keep the catalog current and comprehensive.

CONCLUSION

The **Movie Streaming Application** project successfully addresses the critical need for a centralized, full-featured, and interactive digital streaming platform. By integrating movie management, user authentication, community reviews, and an admin content ecosystem, the platform provides a complete solution for both content administrators and streaming audiences.

The development process demonstrated the power and flexibility of the **MERN Stack** (MongoDB, Express.js 5, React 19, Node.js) enhanced with **TypeScript**. The use of **React 19 with ViteRedux** provided efficient state management for complex features like JWT-based authentication and movie catalog browsing with genre filtering. On the backend, **Node.js** and **MongoDB** proved to be a highly scalable combination, capable of handling diverse data types—from embedded review subdocuments and YouTube video metadata to structured user profiles and genre references.

Key features such as **Genre-based Movie Filtering** for content discovery, **YouTube Video Embedding** for direct content streaming, and the **Star Rating & Review System** for community-driven content evaluation have created an engaging platform where users can discover, watch, and review movies. The implementation of secure authentication using **JWT** and **Bcrypt** ensures that the platform remains safe and trustworthy for all users.

In conclusion, Movie Streaming Application is not just a simple movie list; it is a scalable, modern full-stack streaming solution built with industry-standard technologies. With its solid MERN + TypeScript architectural foundation, comprehensive API documentation via Swagger,

and clear path for future upgrades including AI recommendations and Redis caching, it is poised to evolve into a fully-featured streaming platform.

BIBLIOGRAPHY

The following resources, documentation, and literature were referred to during the design, development, and implementation of this project:

1. Official Documentation & Frameworks

- **Express.js Team.** (2024). *Express 5.x API Reference*. OpenJS Foundation. Retrieved from https://expressjs.com/
- **Meta Platforms, Inc.** (2024). *React 19 Reference Overview*. React.dev. Retrieved from https://react.dev/reference/react
- **Microsoft.** (2024). *The TypeScript Handbook: TypeScript 5.8*. Retrieved from https://www.typescriptlang.org/docs/
- **MongoDB, Inc.** (2024). *Mongoose 8.x: Elegant MongoDB Object Modeling for Node.js*. Retrieved from https://mongoosejs.com/docs/
- **Tailwind Labs.** (2024). *Tailwind CSS v4 Documentation*. Retrieved from https://tailwindcss.com/docs
- **Node.js.** (2024). *Node.js v18.x Documentation*. OpenJS Foundation. Retrieved from https://nodejs.org/docs/

2. Libraries, Tools & APIs

- **Redux Toolkit.** (2024). *RTK Query: Data Fetching and Caching*. Retrieved from https://redux-toolkit.js.org/rtk-query/overview
- **Vite.** (2024). *Vite: Next Generation Frontend Tooling*. Retrieved from https://vitejs.dev/guide/

- **The Movie Database (TMDB).** (2024). *TMDB API Documentation*. Retrieved from https://developer.themoviedb.org/docs
- **Google Developers.** (2024). *YouTube Player API Reference for iframe Embeds*. Retrieved from https://developers.google.com/youtube/iframe_api_reference
- **Swagger.** (2024). *OpenAPI Specification & Swagger UI*. SmartBear Software. Retrieved from https://swagger.io/specification/

3. Theoretical Foundations & Standards

- **Fielding, R. T.** (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine. [Foundational theory for RESTful APIs used in the Controller architecture]
- **IETF.** (2015). *JSON Web Token (JWT) - RFC 7519*. Internet Engineering Task Force. Retrieved from https://tools.ietf.org/html/rfc7519 [Standard used for the createToken.ts authentication utility]
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. [Reference for the MVC pattern used in the backend structure]